ในกรณีทั้งหมดต่อไปนี้ กุญแจส่วนตัวของผู้ใช้คือจำนวนเต็มสุ่ม $d \in [1, n-1]$ และกุญแจสาธารณะที่สอดคล้องกันคือจุด $Q = dG$

4.1 Elliptic Curve Diffie–Hellman (ECDH)

ECDH ช่วยให้คู่สนทนาสองฝ่ายสามารถสร้างความลับร่วมกัน (shared secret) ผ่านช่องทางสาธารณะได้ โดยกระบวนการแลกเปลี่ยนดังกล่าวแสดงไว้ในรูปที่ 3

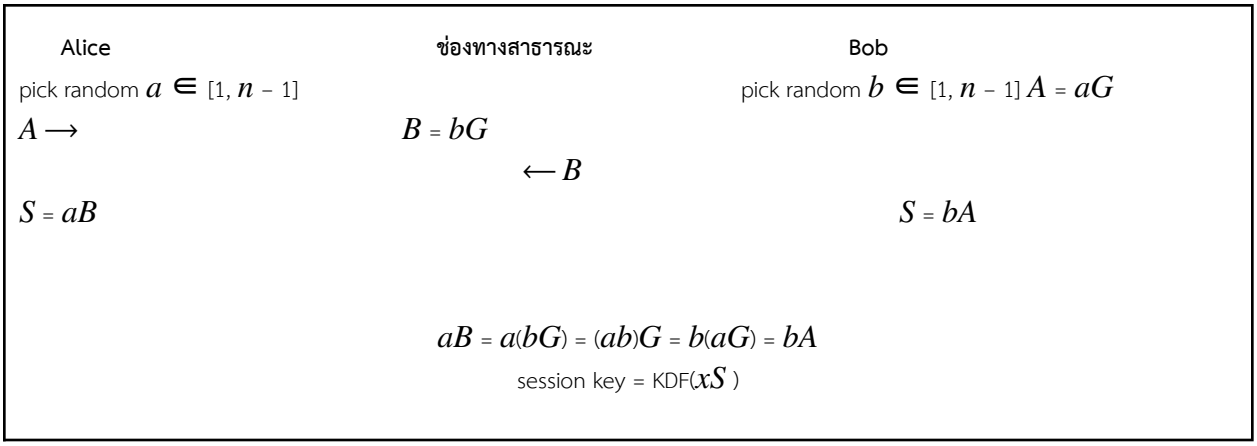


Figure 3: Elliptic Curve Diffie–Hellman key agreement.

ผู้ดักฟังจะเห็นค่า G, A และ B จะต้องคำนวณหาค่า abG ให้ได้ ปัญหานี้เรียกว่าปัญหาการคำนวณแบบดิฟฟี-เฮลล์แมน (Computational Diffe-Hellman problem) ซึ่งเชื่อกันว่ามีความยากเทียบเท่ากับปัญหาลอการิทึมไม่ต่อเนื่องบนเส้นโค้งวงรี (ECDLP) สำหรับเส้นโค้งมาตรฐาน โดยจุดที่ได้ร่วมกันนั้นจะไม่มีการนำไปใช้งานดดยตรง แต่จะนำค่าพิกัด x ของจุดดังกล่าวไปผ่านฟังก์ชั่นสร้างกุญแจ (key-derivation function)

ในกลุ่มตัวอย่างนี้ หาก Alice เลือก $a = 3$ จะได้ $A = (10, 6)$ และหาก Bob เลือก $b = 7$ จะได้ $B = (0, 6)$ ทั้งสองฝ่ายจะได้ค่า $S = 21G = 2G = (6, 3)$

เนื่องจาก ECDH แบบปกติไม่มีกลไกการยืนยันตัวตน ผู้โจมตีแบบเชิงรุก (active attacker) จึงอาจแทรกตัวอยู่ระหว่าง Alice และ Bob ได้ ในทางปฏิบัติ ค่าสาธารณะจะถูกลงนามกำกับไว้ (เช่น ในโปรโตคอล TLS) หรือผูกไว้กับกุญแจระบุตัวตนระยะยาว (เช่น ในโปรโตคอล X3DH ของ Signal)

การใช้กุญแจชั่วคราว(Ephemeral keys) ที่สร้างขึ้นใหม่สำหรับทุกเซสชัน (เรียกว่า ECDHE) จะช่วยให้เกิดคุณสมบัติความลับแบบส่งต่อ (forward secrecy) ซึ่งหมายความว่าหากกุญแจระยะยาวถูกบุกรุกในภายหลัง ข้อมูลของเซสชันที่ผ่านมาก็จะไม่ถูกเปิดเผย ทั้งนี้ โปรโตคอล TLS 1.3 กำหนดให้ต้องมีการแลกเปลี่ยนกุญแจชั่วคราวสำหรับการสร้างการเชื่อมต่อ (Handshake) ทุกครั้งที่ไม่ได้เป็นการกลับมาเชื่อมต่อใหม่ (Resumed session)

4.2 Elliptic Curve Digital Signature Algorithm (ECDSA)

ECDSA ถูกกำหนดมาตรฐานไว้ใน FIPS 186 เป็นอัลกอริทึมที่พัฒนาต่อยอดมาจาก DSA โดยใช้หลักการของเส้นโค้งวงรี กำหนดให้ H เป็นฟังก์ชั่นแฮชเชิงรหัสลับ (Crytographic hash function) และให้ z คือค่าที่ได้จากบิตทางซ้ายสุดจำนวน $[log_2 n]$ บิต ของผลลัพธ์จากฟังก์ชั่นแฮช $H(m)$ สำหรับข้อความ m ใดๆ

อัลกอริทึมที่ 2:การสร้างและการตรวจสอบลายเซ้นดิจิทัลแบบ ECDSA

การสร้างลายเซ็น (Sign) (d, m):

1. เลือกค่าสุ่ม (nonce) $k \in [1, n - 1]$ ขึ้นมาใหม่ โดยให้เป็นค่าความลับและสุ่มแบบกระจายตัวสม่ำเสมอ
2. คำนวณหา $(x_1, y_1) = kG$ และกำหนดให้ $r = x_1 \bmod n$ หาก $r = 0$ ให้ย้อนกลับไปทำขั้นตอนที่ 1 ใหม่
3. คำนวณหา $s = k^{-1}(z + rd) \bmod n$ หาก $s = 0$ ให้ย้อนกลับไปทำขั้นตอนที่ 1 ใหม่
4. ส่งผลลัพธ์เป็นลายเซ็น (r, s)

การตรวจสอบลายเซ็น (Verify) ($Q, m, (r, s)$):

1. ตรวจสอบว่า $r, s \in [1, n - 1]$ และ Q เป็นจุดที่ถูกต้องบนเส้นโค้งซึ่งมีอันดับ (order) เท่ากับ n
2. คำนวณหา $w = s^{-1} \bmod n$, $u_1 = zw \bmod n$ และ $u_2 = rw \bmod n$
3. คำนวณหา $(x_1, y_1) = u_1G + u_2Q$ หากผลลัพธ์ที่ได้คือจุดอนันต์ ($\mathcal{O}$) ให้ปฏิเสธลายเซ็นนี้
4. ยอมรับลายเซ็นก็ต่อเมื่อ $r \equiv x_1 \pmod n$ เท่านั้น

ความถูกต้องเป็นผลมาจาก $u_1G + u_2Q = s^{-1}(z + rd)G = kG$

ค่า nonce จะต้องถูกเก็บเป็นความลับและห้ามนำกลับมาใช้ซ้ำเด็ดขาด หากลายเซ็นสองลายเซ็น (r, $s1$) และ (r, $s2$)ในข้อความที่แตกต่างกันใช้ nonce k ร่วมกัน (ซึ่งมองเห็นได้เป็น r ที่เท่ากัน) ใครก็ตามก็สามารถกู้คืนคีย์ส่วนตัวได้:

$$k = \frac{z_1 - z_2}{s_1 - s_2} \bmod n, d = \frac{s_1 k - z_1}{r} \bmod n$$

นี่คือวิธีเดียวกับที่คีย์สำหรับลงนามโค้ด (code-signing key) ของ PlayStation 3 ถูกดึงออกมาในปี ค.ศ. 2010 โดยในการใช้งานนั้นมีการใช้ค่าคงที่ k ซึ่งเพียงแค่บิตบางส่วนของค่า nonce ที่รั่วไหลออกมา ก็เพียงพอต่อการกู้คืนคีย์ด้วยเทคนิคที่อาศัยโครงสร้างแบบแลตทิซ (lattice-based key recovery) ทั้งนี้ มาตรฐาน RFC 6979 ได้ขจัดปัญหาการต้องพึ่งพาตัวสร้างตัวเลขสุ่ม โดยใช้วิธีคำนวณค่า k แบบกำหนดผลลัพธ์ตายตัว (deterministically) จากค่า d และ $H(m)$ แทน

4.3 EdDSA and Ed25519

EdDSA เป็นรูปแบบลายเซ็นดิจิทัลในสไตล์ Schnorr ซึ่งสร้างขึ้นบนเส้นโค้ง Edwards แบบบิดเบี้ยว (twisted Edwards curves) โดยได้รับการออกแบบมาเพื่อขจัดข้อบกพร่องต่างๆ ที่พบใน ECDSA สำหรับ Ed25519 ซึ่งเป็นรูปแบบที่รู้จักกันมากที่สุดของ EdDSA นั้น มีกระบวนการทำงานดังนี้: เริ่มต้นด้วยการนำค่าตั้งต้นที่เป็นความลับ (secret seed) ขนาด 32 ไบต์ มาผ่านฟังก์ชั่นแฮช SHA-512 ผลลัพธ์ที่ได้จะถูกแบ่งออกเป็นสองส่วน ส่วนแรกใช้เป็นค่าสเกลาร์ลับ (secret scalar) a โดยที่กุญแจสาธารณะคือ $A = aB$(เมื่อ B คือจุดฐาน หรือ base point) ส่วนอีกครึ่งหนึ่งจะถูกใช้เป็นค่าคำนำหน้าลับ (secret prefix) สำหรับการลงลายเซ็นในข้อความ M มีขั้นตอนดังนี้:

$$r = H(\text{prefix} \parallel M), R = rB, S = (r + H(R \parallel A \parallel M) \cdot a) \bmod \ell,$$

โดยที่ ℓ คืออันดับของกลุ่มที่เป็นจำนวนเฉพาะ ลายเซ็นคือ (R, S) ผู้ตรวจสอบจะทำการตรวจสอบว่า $8SB = 8R + 8H(R \parallel A \parallel M)A$

เมื่อเปรียบเทียบกับ ECDSA แล้ว EdDSA มีข้อได้เปรียบในทางปฏิบัติหลายประการ:

• nonce เป็นแบบกำหนดได้ ดังนั้นเครื่องกำเนิดเลขสุ่มที่อ่อนแอจึงไม่สามารถรั่วไหลคีย์ได้

• การลงนามไม่จำเป็นต้องใช้การผกผันโมดูลาร์

• กฎการบวกของ Edwards ที่สมบูรณ์ไม่มีกรณีพิเศษ ซึ่งทำให้โค้ดเวลาคงที่ตรงไปตรงมา

• การตรวจสอบลายเซ็นจำนวนมากสามารถรวมกลุ่มกันเพื่อเพิ่มความเร็ว

Bitcoin ได้นำลายเซ็น BIP 340 Schnorr ที่เกี่ยวข้องอย่างใกล้ชิดมาใช้แทน secp256k1 ในการอัปเกรด Taproot ในปี 2021

4.3 Elliptic Curve Integrated Encryption Scheme (ECIES)

โดยทั่วทั่วไปแล้ว ECC ไม่ค่อยถูกนำมาใช้ในการเข้ารหัสข้อมูลดดยตรง แต่จะใช้วิธีการเข้ารหัสแบบผสม (Hybrid Encryption) เช่น ECIES ซึ่งเป็นการรวมกระบวนการแลกเปลี่ยนกุญแจแบบ ECDH ชั่วคราว (Ephemeral ECDH) เข้ากับการเข้ารหัสแบบสมมาตร (Symmetric Cryptography) โดยผู้ส่งจะสร้างคู่กุญแจชั่วคราวขึ้นมา (e, $E = eG$) และคำนวณค่า $S = eQ_$recipient จากนั้นนำค่า S ไปผ่านฟังก์ชันสร้างกุญแจ (Key Derivation Function หรือ KDF) เพื่อให้ได้กุญแจสำหรับการเข้ารหัสและกุญแจสำหรับ MAC แล้วจึงส่งค่า E ไปพร้อมกับข้อมูลที่เข้ารหัสซึ่งมีการตรวจสอบความ

ถูกต้อง (Authenticated Ciphertext) ทางด้านผู้รับจะคำนวณค่า S ใหม่ด้วยสูตร $S = dE$ รูปแบบเดียวกันนี้ยังเป็นพื้นฐานของ HPKE (ตามมาตรฐาน RFC 9180) ซึ่งถูกนำไปใช้ใน TLS Encrypted Client Hello และ Messaging Layer Security อีกด้วย